

LINUX NETWORKING SERVICES PLAYBOOK

3

Prepared By: Hosni Zaaraoui

Introduction: Optimizing Traffic and Enhancing Performance

In modern network architectures, efficiently managing traffic and accelerating content delivery are critical for maintaining high availability, security, and performance. This dual-sectioned part of the playbook covers two pivotal components of network infrastructure:

- **Load Balancers:** The traffic directors that distribute workloads across servers to prevent overloads, ensure redundancy, and optimize resource use.
- **Proxy Servers:** The intermediaries that cache content, filter requests, and add layers of security and privacy.

Why These Services Matter

- **Scalability:** Handle growing traffic seamlessly by distributing loads or caching frequent requests.
- **Reliability:** Keep applications online even during server failures or traffic spikes.
- **Speed:** Reduce latency via caching (proxies) or optimized routing (load balancers).
- **Security:** Block malicious traffic, hide server identities, and enforce access policies.

LOAD BALANCERS

1. Introduction to Load Balancers

What is a Load Balancer?

A load balancer distributes incoming network traffic across multiple servers to optimize resource utilization, maximize throughput, minimize response time, and ensure high availability. It acts as a "traffic cop," preventing any single server from becoming overloaded while maintaining seamless user experiences.

Why are Load Balancers Important?

- **High Availability:** Ensures applications remain online even if a server fails.
- **Scalability:** Distributes traffic to handle increased load without downtime.
- **Performance:** Reduces latency by routing requests to the nearest/least busy server.
- **Security:** Can filter malicious traffic and provide SSL termination.

Types of Load Balancing

1. **Layer 4 (Transport Layer):** Balances traffic based on IP and port (e.g., HAProxy in TCP mode).
2. **Layer 7 (Application Layer):** Routes traffic based on HTTP headers, URLs, or cookies (e.g., Nginx).
3. **Global Server Load Balancing (GSLB):** Distributes traffic across geographically dispersed servers.

2. HAProxy: A High-Performance Load Balancer

HAProxy is a reliable, open-source solution ideal for TCP/HTTP load balancing. It's widely used for its speed and flexibility.

Step 1: Install HAProxy

On Debian/Ubuntu:



```
sudo apt update
sudo apt install haproxy
```

On CentOS/RHEL:



```
sudo yum install haproxy
```

Step 2: Configure HAProxy

Edit `/etc/haproxy/haproxy.cfg`:

– Basic HTTP Load Balancing



```
frontend http_front
    bind *:80
    default_backend http_back

backend http_back
    balance roundrobin
    server server1 192.168.1.10:80 check
    server server2 192.168.1.11:80 check
```

– SSL Termination (HTTPS)

```
frontend https_front
    bind *:443 ssl crt /etc/ssl/certs/your-cert.pem
    default_backend http_back
```

Key Directives Explained

- **balance roundrobin**: Distributes requests equally (other algorithms: **leastconn**, **source**).
- **check**: Enables health checks to mark servers as UP/DOWN.
- **ssl crt**: Specifies SSL certificates for HTTPS.

Step 3: Start and Enable HAProxy

```
sudo systemctl restart haproxy
sudo systemctl enable haproxy
```

Step 4: Monitor HAProxy

View real-time stats by enabling the stats page:

```
1 listen stats
2     bind *:8404
3     stats enable
4     stats uri /stats
5     stats refresh 30s
```

Access at <http://<your-server-ip>:8404/stats>.

3. Nginx: The Versatile Load Balancer

Nginx doubles as a web server and load balancer, excelling at Layer 7 (HTTP/HTTPS) balancing.

Step 1: Install Nginx

On Debian/Ubuntu:



```
sudo apt install nginx
```

On CentOS/RHEL:



```
sudo yum install nginx
```

Step 2: Configure Nginx for Load Balancing

Edit `/etc/nginx/nginx.conf`:

– Basic HTTP Load Balancing



```
1 http {
2     upstream backend {
3         server 192.168.1.10;
4         server 192.168.1.11;
5     }
6
7     server {
8         listen 80;
9         location / {
10             proxy_pass http://backend;
11         }
12     }
13 }
```

– Weighted Load Balancing



```
1 upstream backend {  
2     server 192.168.1.10 weight=3; # Handles 3x more traffic  
3     server 192.168.1.11 weight=1;  
4 }
```

– Health Checks



```
1 server 192.168.1.10 max_fails=3 fail_timeout=30s;
```

Step 3: Start Nginx



```
sudo systemctl restart nginx  
sudo systemctl enable nginx
```

4. Advanced Load Balancing Techniques

Session Persistence (Sticky Sessions)

- HAProxy: Use **cookie** directive:



```
backend http_back  
    cookie SERVERID insert  
    server server1 192.168.1.10:80 cookie s1  
    server server2 192.168.1.11:80 cookie s2
```

- Nginx: Use **ip_hash**:

```
upstream backend {  
    ip_hash;  
    server 192.168.1.10;  
    server 192.168.1.11;  
}
```

SSL Offloading

Terminate SSL at the load balancer to reduce backend server load:

- **HAProxy**: As shown earlier with **ssl crt**.
- **Nginx**:

```
server {  
    listen 443 ssl;  
    ssl_certificate /path/to/cert.pem;  
    ssl_certificate_key /path/to/key.pem;  
    location / {  
        proxy_pass http://backend;  
    }  
}
```

5. Troubleshooting Load Balancers

– Common Issues

1. Servers Marked DOWN

- Verify backend servers are reachable (**curl -I http://192.168.1.10**).

- Check HAProxy/NGinx logs:

```
sudo tail -f /var/log/haproxy.log
sudo journalctl -u nginx
```

2. High Latency

- Use leastconn algorithm (HAProxy) or adjust weights (NGinx).
- Monitor server resources (CPU/RAM) with top or htop.

3. SSL Errors

- Ensure certificates are valid and paths are correct.
- Test with `openssl s_client -connect your-domain:443`.

6. Best Practices

- **Health Checks:** Always enable them to avoid routing traffic to failed servers.
- **Redundancy:** Deploy multiple load balancers in active-passive mode.
- **Logging:** Centralize logs for monitoring (e.g., ELK Stack).
- **Security:** Restrict access to load balancer ports (80/443) and stats pages.

7. Conclusion

Load balancers are the unsung heroes of high-traffic systems, ensuring scalability and reliability. Whether you choose HAProxy for raw performance or NGinx for its HTTP prowess, mastering these tools will elevate your infrastructure.

PROXY SERVERS AND CACHING

1. Introduction to Proxy Servers and Caching

What is a Proxy Server?

A proxy server acts as an intermediary between clients (users) and other servers. It forwards requests, caches content, filters traffic, and enhances security/privacy.

Why Use a Proxy Server?

- **Bandwidth Savings:** Caches frequently accessed content, reducing bandwidth usage.
- **Security:** Filters malicious traffic and hides internal network structures.
- **Access Control:** Restricts access to specific websites or services.
- **Performance:** Accelerates web requests via caching.

Types of Proxy Servers

1. **Forward Proxy:** Sits between clients and the internet (e.g., Squid).
2. **Reverse Proxy:** Sits between servers and clients (e.g., Nginx, HAProxy).
3. **Transparent Proxy:** Intercepts traffic without client configuration.

2. Squid: The Classic Forward Proxy

Squid is a robust, open-source proxy server supporting HTTP, HTTPS, and FTP.

Step 1: Install Squid

On Debian/Ubuntu:



```
sudo apt update && sudo apt install squid
```

On CentOS/RHEL:



```
sudo yum install squid
```

Step 2: Configure Squid

Edit `/etc/squid/squid.conf`:

– Basic Forward Proxy



```
1 http_port 3128
2 acl localnet src 192.168.1.0/24 # Allow internal network
3 http_access allow localnet
4 http_access deny all
5 cache_dir ufs /var/spool/squid 100 16 256 # Enable caching
```

Key Directives

- **http_port**: Sets the proxy listening port.
- **acl**: Defines access control lists (IP-based rules).
- **cache_dir**: Configures caching (size in MB, subdirectories).

Step 3: Start Squid



```
sudo systemctl restart squid  
sudo systemctl enable squid
```

Step 4: Test the Proxy

Configure a client to use the proxy (**192.168.1.10:3128**) and verify:



```
curl --proxy http://192.168.1.10:3128 -I http://example.com
```

3. Advanced Squid Configurations

Access Control

– Block Websites:



```
acl banned_sites dstdomain .facebook.com .twitter.com  
http_access deny banned_sites
```

– Time-Based Restrictions:



```
acl work_hours time MTWHF 09:00-17:00  
http_access deny !work_hours
```

Caching Optimization

– Cache Size Tuning:



```
cache_mem 256 MB # RAM for hot objects  
maximum_object_size 50 MB # Skip caching large files
```

– Refresh Patterns:



```
refresh_pattern .html$ 1440 20% 10080
```

Logging

Monitor access logs in real-time:



```
tail -f /var/log/squid/access.log
```

4. Reverse Proxy Caching with Nginx

Nginx can cache responses from backend servers to reduce load.

Step 1: Enable Caching in Nginx

Edit `/etc/nginx/nginx.conf`:

```
1 proxy_cache_path /var/cache/nginx levels=1:2 keys_zone=my_cache:10m
   inactive=60m;
2
3 server {
4     location / {
5         proxy_pass http://backend;
6         proxy_cache my_cache;
7         proxy_cache_valid 200 302 10m;
8     }
9 }
```

Step 2: Purge Cache

Manually delete cached files:

```
rm -rf /var/cache/nginx/*
```

5. Troubleshooting Proxy Servers

– Common Issues

1. Clients Can't Connect

- Verify Squid/Nginx is running:



```
sudo systemctl status squid
```

- Check firewall rules (**iptables -L**).

2. Slow Performance

- Increase cache size or adjust refresh_pattern.
- Monitor disk I/O with **iostat**.

3. SSL Errors

- For HTTPS proxying, configure SSL bumping in Squid or use Nginx for SSL termination.

6. Best Practices

– Security:

- Restrict proxy access to trusted IPs.
- Use SSL/TLS for sensitive traffic.

– Monitoring:

- Use tools like squid-analyzer for traffic analysis.

– Maintenance:

- Regularly rotate logs (logrotate).
- Update Squid/Nginx to patch vulnerabilities.

7. Conclusion

Proxy servers and caching strategies are vital for optimizing performance, controlling access, and securing networks. Whether you use Squid for forward proxying or Nginx for reverse caching, these tools empower you to build efficient, scalable infrastructures.

